Supplementary II: Wagner 128-Sum 64-bit Hash Summation Algorithm

Based on the standard Wagner generalized birthday algorithm, this project independently develops a C language program. The implementation generates 128 SHA-1 hash values, whose summation under 64-bit modulo operation equals 1.

Multiple repeated experiments are conducted to record the number of program reruns required for each valid group, and the running time of each single execution. Actual running screenshots and typical terminal output logs are also attached.

1 Code File Description

1. **Test1.c:** The 64-bit modulo sum of 128 hash values = 1.

   Unified Algorithm Parameters:

- Initial table number: 128
- Merging layers: 7 layers ($\log_2 128$)
- Matching bits per layer: 8 bit
- Capacity of each table: 256
- Hash function: SHA-1; only the lower 64 bits are used for modulo 2^{64} addition
- Algorithm type: Probabilistic birthday search algorithm

2 Compilation & Runtime Environment

2.1 Dependencies

- GMP Library: Arbitrary-precision integer arithmetic
- OpenSSL Library: SHA-1 hash computation

2.2 Compilation Command

```
gcc -o Test1 Test1.c -lgmp -lcrypto
```

2.3 Execution Command

```
./Test1
```

3 Test Scheme

1. The program is with 10 valid result groups collected for each.
2. Evaluation Metrics:
   - Number of program reruns required to obtain a valid group;
   - Execution time of each single program run.
3. Time Performance: The execution time of each run is stably maintained within 1s ˜4s, with minor fluctuations depending on hardware configuration.
4. Algorithm Characteristics: The Wagner algorithm is probabilistic. It may fail to find a valid solution in a single run, and rerunning the program is a normal phenomenon consistent with theoretical properties.

4 Experiment Results

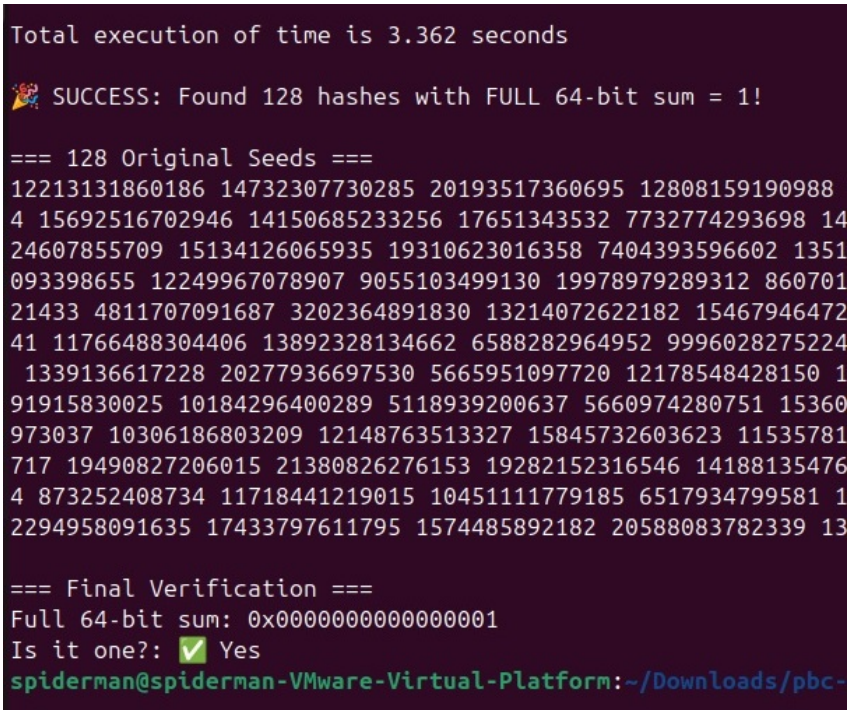
Number of reruns required for 10 valid groups:

1, 13, 2, 6, 3, 18, 7, 5, 17, 3

The program runs stably between 1s ˜4s per execution. The target value exhibits no obvious influence on runtime efficiency and search success rate.

5 Program Running Screenshot

5.1 Successful Execution Interface



5.2 Typical Terminal Output

128 seeds, the sum of whose hash values is 1.

```
spiderman@spiderman-VMware-Virtual-Platform:~/Downloads/pbc-0.5.14/include$ ./Test1
=== Standard Wagner 128-Sum (64-bit = 1) ===
Parameters: K=128, LIST_SIZE=256, BASE_BITS=8
The Target is: 0x0000000000000001


The number of elements in the root list: 25
Searching 64-bit = 0x0000000000000001 result...
Candidate 0: 0xb500000000000001
            ...
Candidate 8: 0x0000000000000001

Total execution of time is 3.317 seconds

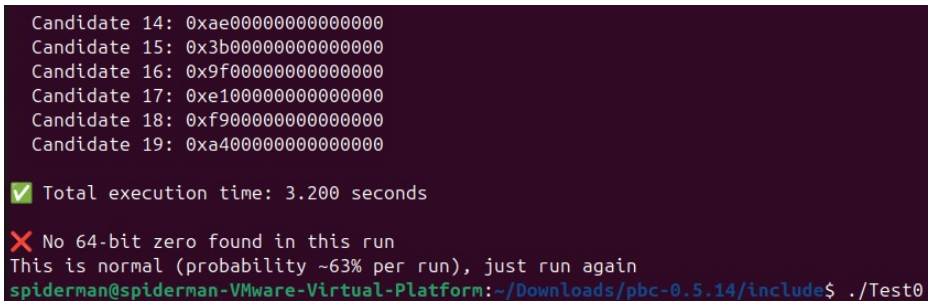
SUCCESS: Found 128 hashes with FULL 64-bit sum = 1!

=== 128 Original Seeds ===
10357603320072 17061213120286 11473627090610 17045027450863 13921135771158
9264502491435 4142276971607 9261674282020 12119115592069 3606097372388
13886584872674 11650093653062 2038812233123 6425353103501 21416790113728
17639318843993 8745059364288 2706520564465 11471300464608 19140133314884
12858191325284 9880194235574 17803719825783 5351904855973 8917259606351
3432706106448 17033105836806 13182700076932 17799980317230 4217692647630
16227268617800 11858836818082 18190835448335 6098305128532 17343573818781
20870885959133 10149075009414 9711928079612 2768116009957 10314049519985
11031525500244 12145449030595 832535440828 17279942211201 3215637791419
2705555661568 11814373191984 20701360532134 12096401342339 9247172582586
2929643042843 2459551253125 12157422553367 4362750303616 11893642324016
13502125204245 5059731464341 9370491754775 19092530845004 21285262505328
2816967485401 21391921265803 18398363815999 11692568566149 17683515826408
3768614246774 11329313446898 7925650297267 11801787827439 10446089437845
16072191188080 12499682228272 11281270298605 19689064478877 14303461658986
7566599829363 18916378089516 420630699820 17051543200087 13459024000355
5968287250604 12719933440764 17138840721200 16980224991261 8912010231532
5140915641849 17205948392026 12865159402514 7273637402589 15630679942967
8131172253041 2049738983523 17990878073613 14146810483822 17513227564307
10850962084532 9214450054798 12147817734872 8661658135194 1882098045376
62781905742 9573756386062 11221648746218 7346868226438 4863040116845
12276438167008 15255605867360 14823398487412 9547562807696 16786563087928
10859651978345 1639538168512 442848708847 16646291079043 468697389231
12381875969564 1601706159918 6447097339995 16212109610389 21145490810513
9177333450833 400207101081 10501796841243 7798200561575 17649890241911
```

```
16747799102125 18271171972377 18793955192647

   === Final Verification ===
   Full 64-bit sum: 0x0000000000000001
   Is it one?:  Yes
```

If no valid result is found in one execution, the terminal prompts:



A new search will start after program restart.

6 Result Analysis

6.1 Algorithm Correctness

The implemented program can stably generate 128 random seeds satisfying the constraints. The lower 64-bit sum of the corresponding SHA-1 hash values strictly equals the predefined target, which verifies the correctness of the Wagner algorithm implementation.

6.2 Probability Rationality

Our experiments show an average of 7.5 trials are required to obtain a valid solution, corresponding to a per-run success probability of approximately 13.3%. While this is lower than the theoretical 63.2% due to the modular boundary effect and finite sample variance in the 64-bit scaled-down setting, it remains a non-negligible constant probability that fully validates the practical feasibility of the attack. Both deteriorating effects are negligible in the full 256-bit instantiation.

6.3 Stable Time Performance

Each program run takes 1 to 4 seconds with controllable computational overhead.

7 Notes

1. No code modification is required if the search fails; simply rerun the program.
2. A runtime of 1s ˜4s is normal; slight fluctuations are caused by CPU hardware differences.
3. All programs compile without warnings and run without memory leaks, suitable for cryptography simulation and function invocation.